PipeFusion: Patch-level Pipeline Parallelism for Diffusion Transformers Inference

James, Sankalp

Table of Contents

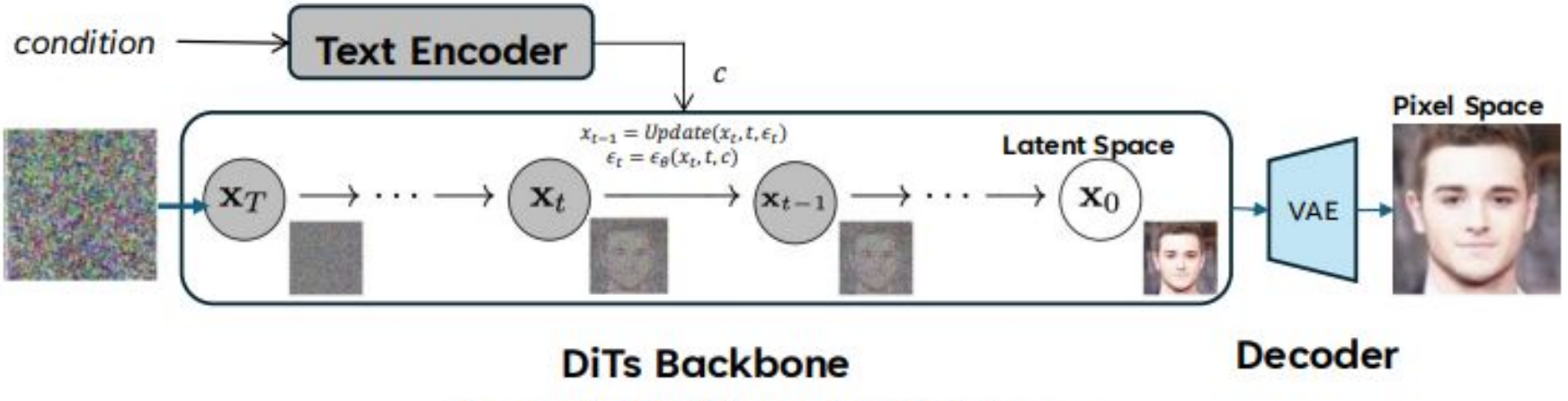
Motivation

- Diffusion models are shifting from U-Nets to DiTs.
- Self-attention scales quadratically with sequence length.
- At large DiT context lengths, a single GPU is too slow for practical serving.

An effective parallelization strategy must focus not only on distributing the computation but also on minimizing communication overhead.

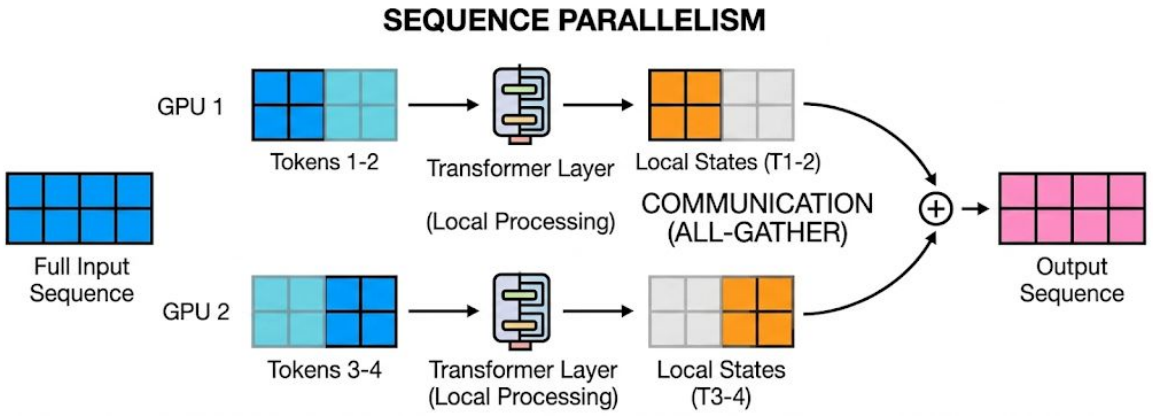
DiT architecture refresher

- Diffusion models generate images by starting with Gaussian noise and iteratively denoising it through time steps.
- DiTs split the noisy latent representations into patches, embed them as tokens, then process them through transformer blocks.

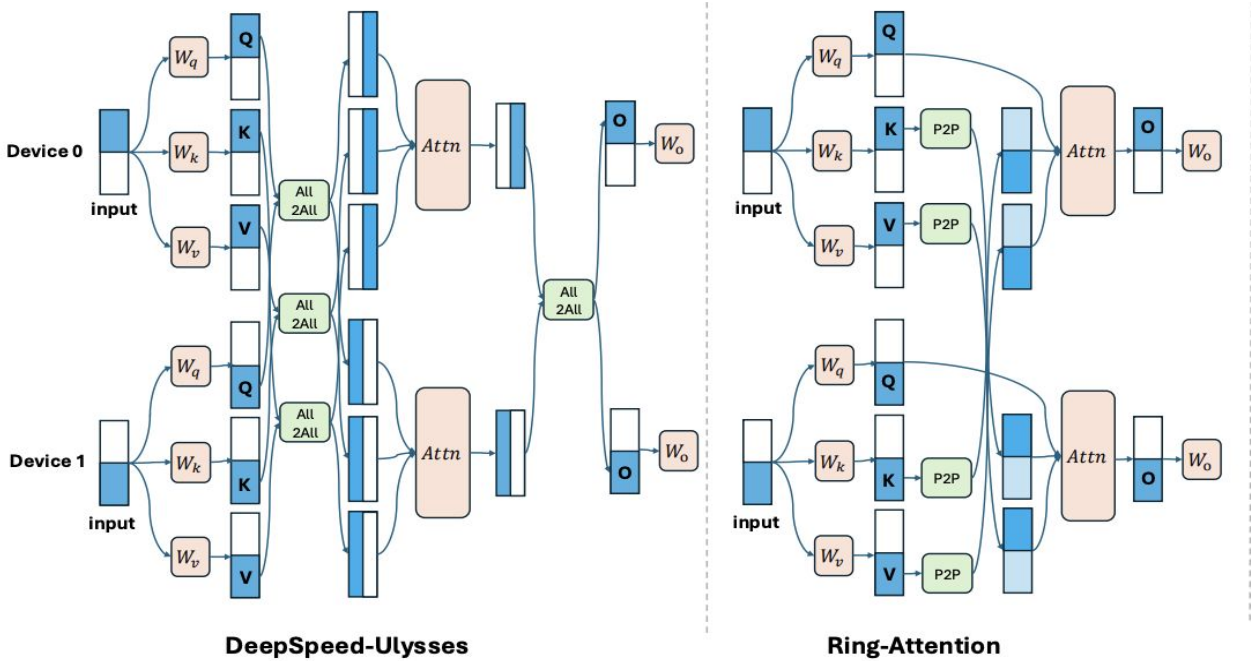


The Gaps in Existing Parallelism

- **Tensor Parallelism (TP):** Partitions linear layers across GPUs, but is inefficient for DiTs because of their massive activation volumes
- **Sequence Parallelism (SP):** Partitions the input image into patches. However, devices still need constant data exchange during attention operations.
- **Asynchronous SP:** Reduces some communication overhead, but scales poorly in memory because it must maintain large K and V states across layers.

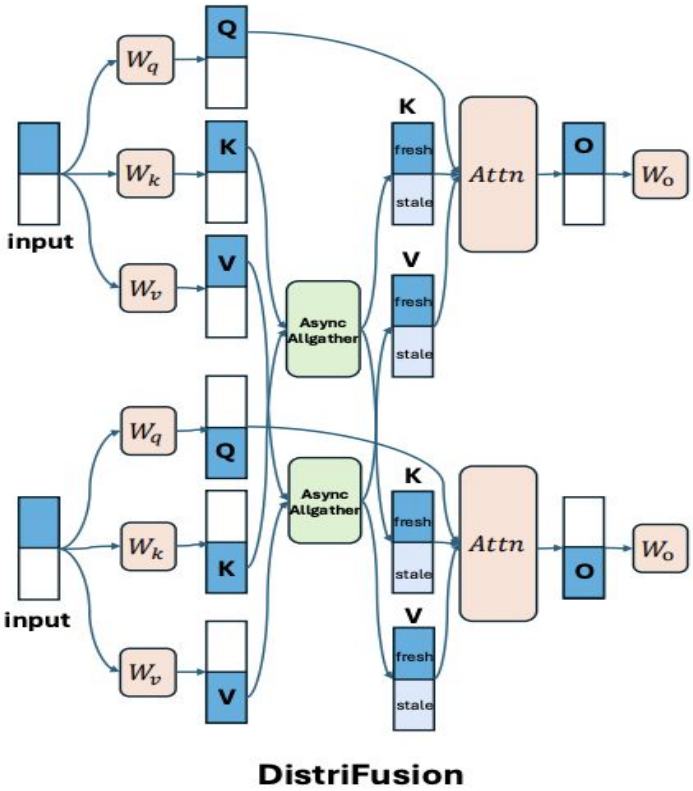


Existing SP Methods



- Ulysses uses All2All communication
- Ring-Attention circulates KV blocks as a ring with P2P transfers around devices.
- Both still communicate inside every attention layer

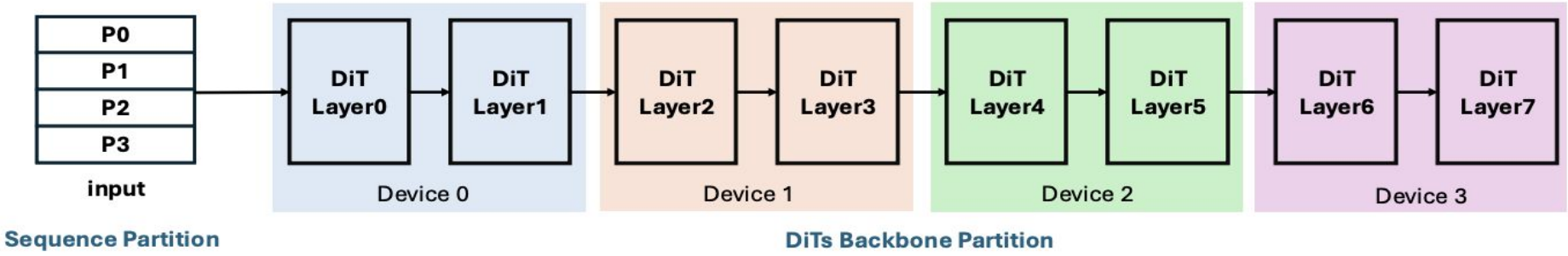
Asynchronous SP: Distrifusion



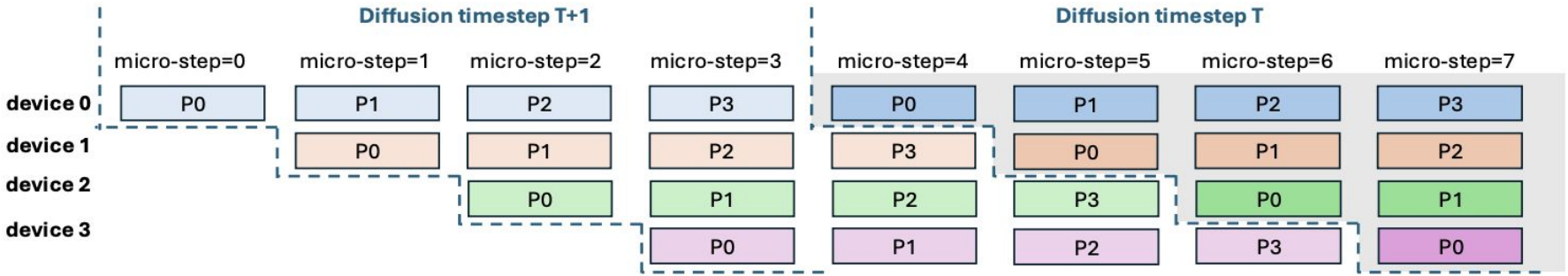
- Uses asynchronous all-gather and stale remote KV from the previous timestep.
- Overlaps communication with the next step's compute.
- But full-spatial KV buffers for all layers make memory costly.

PipeFusion

- **Dual Partitioning:** PipeFusion simultaneously partitions the input sequence (images into non-overlapping patches) and the layers of the DiTs backbone across multiple GPUs.
- **Breaking the Pipeline Bubble:** PipeFusion leverages input temporal redundancy to remove the idle "bubbles" typically caused by devices waiting for data from previous stages



Patch level Pipelined Exectuion



Micro-steps

Each GPU processes one patch through its local stage, then forwards activations to the next device.

Pipeline Fill

Bubbles appear only at startup; after initialization, all stages stay busy.

Recommended Setting

The paper recommends matching patches M to parallel degree N.

Pipefusion vs Distrifusion

- PipeFusion gradually increases the amount of fresh activation within each timestep.
- DistriFusion maintains a fixed small fresh region.
- PipeFusion therefore stays closer to the original diffusion model.

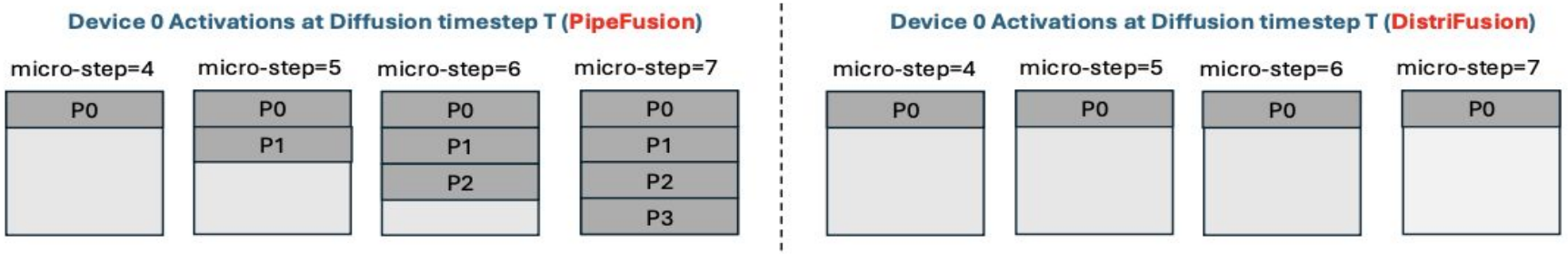


Figure 4: The fresh part of activations during diffusion timestep T of Figure 3. The dark gray represents fresh data and the light gray represents stable data.

Comparison Between Different Parallelisms

- PipeFusion achieves the lowest theoretical communication cost and is independent of the number of layers *L*, unlike other methods.
- It also successfully overlaps this communication with the model's forward computation.

Method	Communication		Memory Cost	
	Cost	Overlap	Model	KV Activations
Tensor Parallel	$4O(p \times hs)L$	✖	$\frac{1}{N}P$	$\frac{1}{N}KV$
DistriFusion	$2O(p \times hs)L$	✔	P	$(KV)L$
SP-Ring	$2O(p \times hs)L$	✔	P	$\frac{1}{N}KV$
SP-Ulysses	$\frac{4}{N}O(p \times hs)L$	✖	P	$\frac{1}{N}KV$
PipeFusion	$2O(p \times hs)$	✔	$\frac{1}{N}P$	$\frac{1}{N}(KV)L$

Latency Evaluation Pt. 1

Pixart: Figure 5 shows the inference latency of Pixart on image generation tasks with resolution as 1024px (1024×1024), 2048px (2048×2048) and 4096px (4096×4096) on 8×L40.

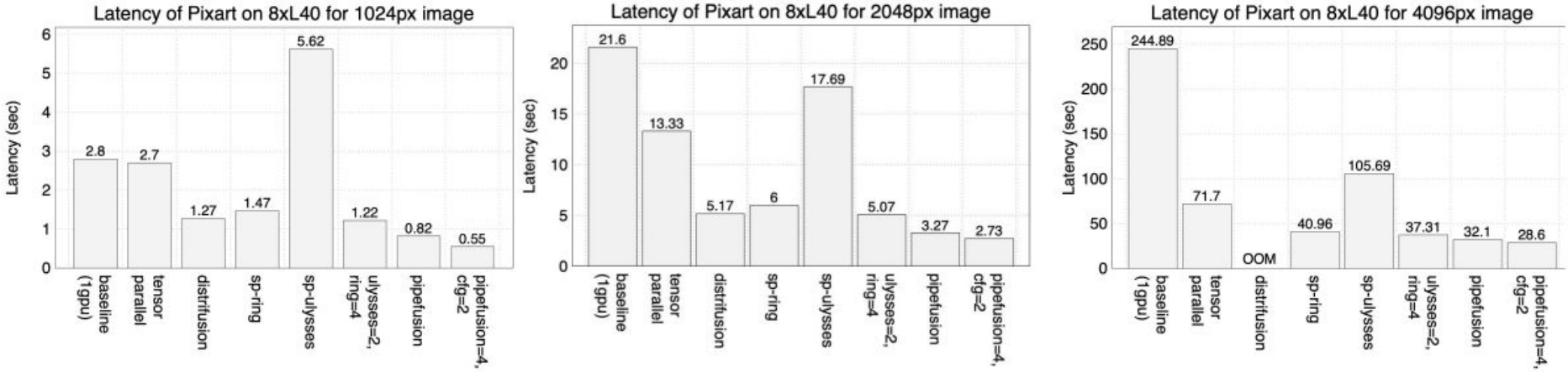
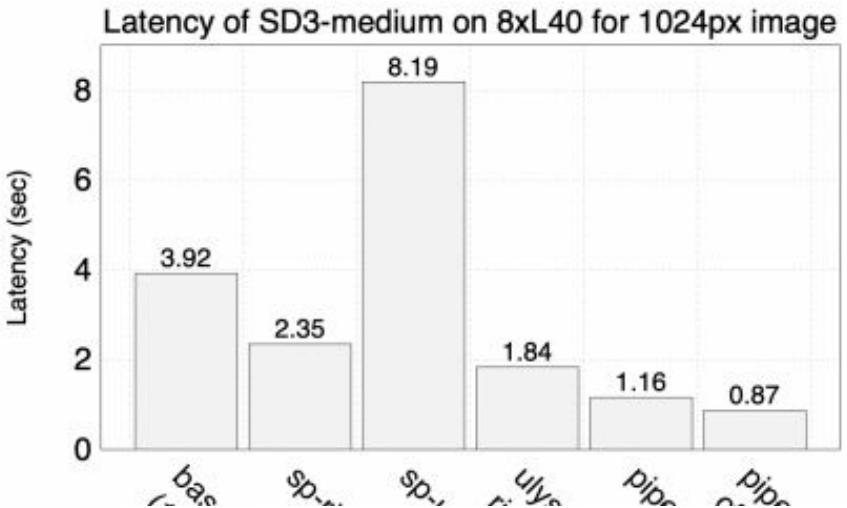


Figure 5: Latency on Pixart of various parallel approaches on two image generation tasks with the 20-Step DPMSolverMultistepScheduler.

Latency Evaluation Pt. 2



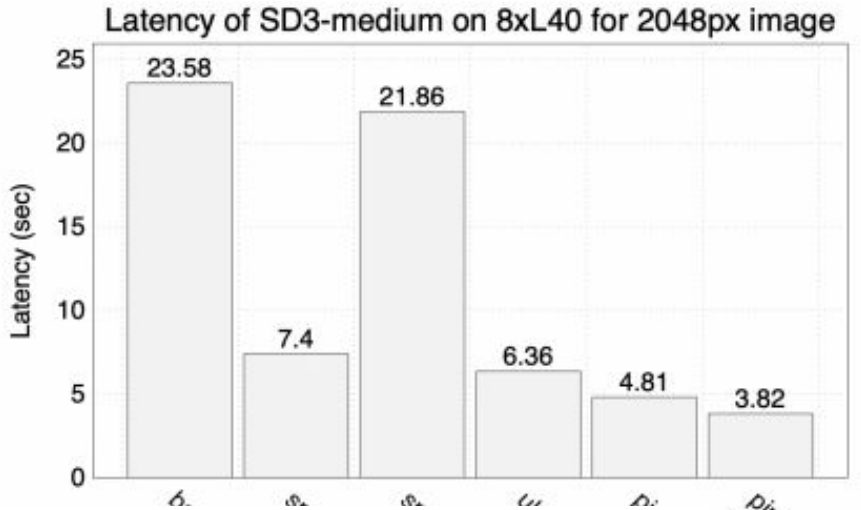


Figure 6: Latency on SD3-medium of various parallel approaches on two image generation tasks with the 20-Step FlowMatchEulerDiscrete Scheduler.

Scalability Analysis of Latency

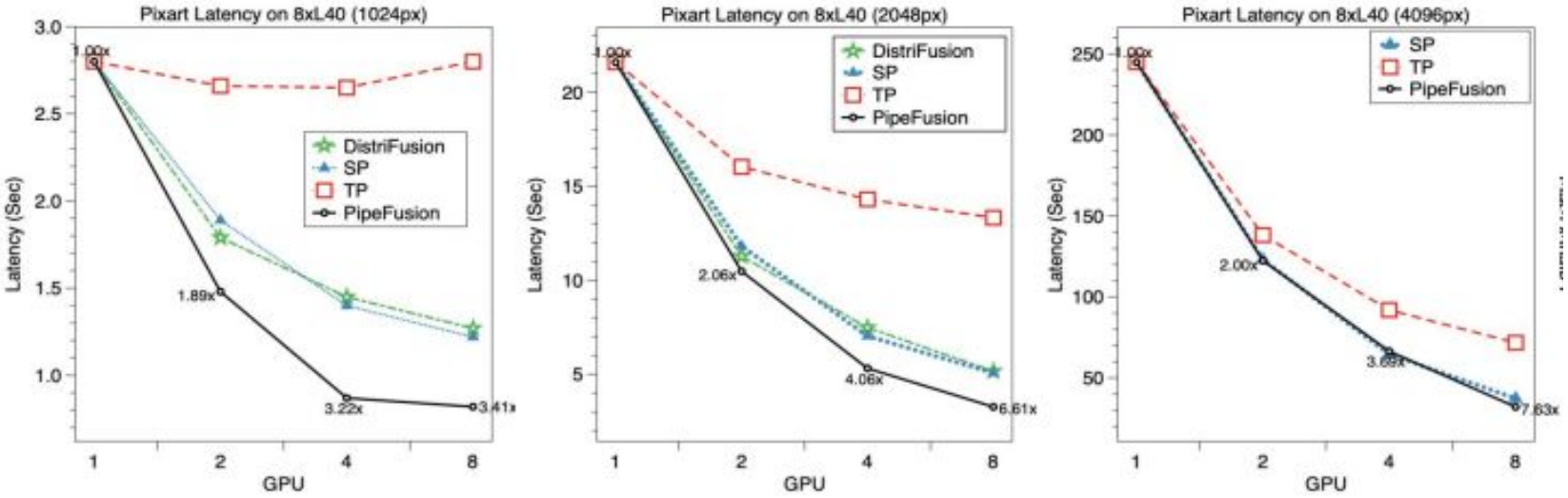
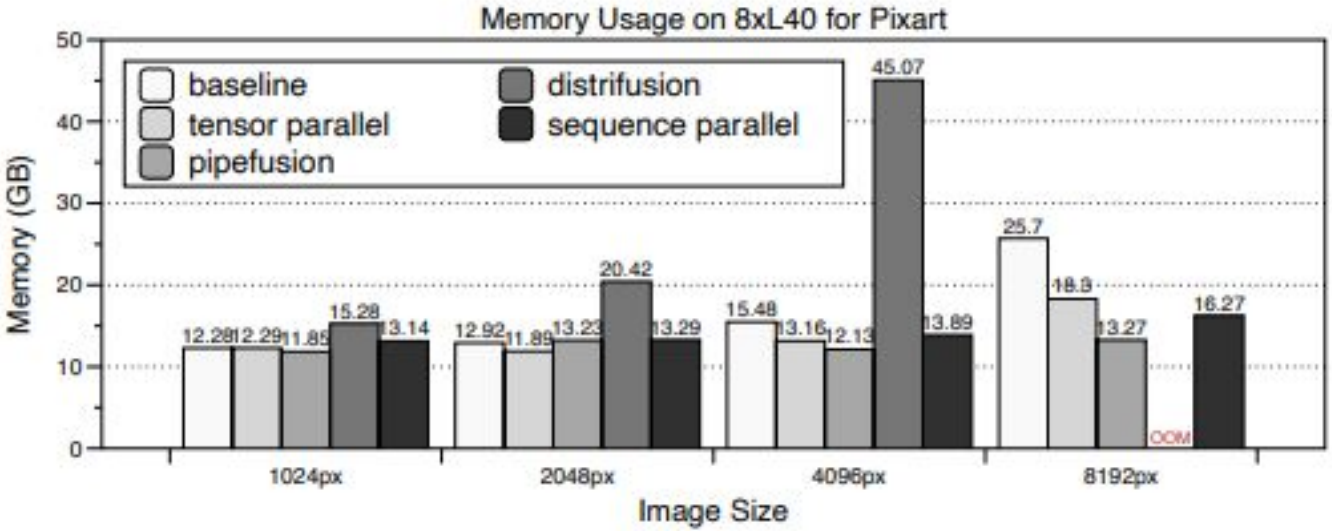


Figure 8: Scalability Analysis of Three DiTs on 8×L40: Leftmost figures depict Pixart performance on 1024px, 2048px, and 4096px images.

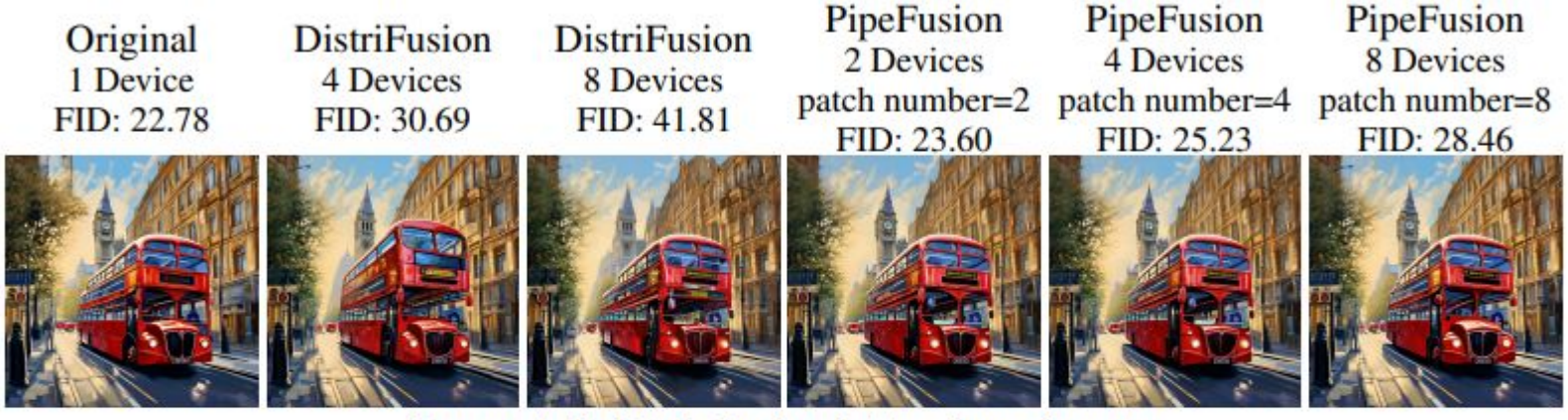
Memory Efficiency Pt. 1



(a) Max GPU Memory Usage of Various Approaches for Pixart Image Generation Tasks Across Four Resolutions.

Results and Takeaways

Model		PixArt-XL-2-256-MS						FLUX.1[dev]		
Method		Original	DistriFusion		PipeFusion			Original	PipeFusion	
Device Number		1	4	8	2	4	8	1	4	8
FID ↓	w/ G.T.	22.78	30.69	41.81	23.60	25.23	28.46	25.03	24.17	25.97
	w/ Orig.	-	8.41	19.81	1.67	3.11	6.09	-	4.01	5.93



Limitations and Tradeoffs

- **Hybrid Deployment Strategy:** PipeFusion is most practical when combined into hybrid schemes that use sequence parallelism for high-bandwidth NVLink connections and PipeFusion for low-bandwidth, inter-node scaling.
- **Minimal gains on Distilled Models:** For one-step or 4-step distilled models, temporal redundancy is minimal, which makes PipeFusion significantly less effective
- **Warmup Phase:** Because temporal redundancy is low during the initial diffusion timesteps, PipeFusion requires a short "warmup" phase of synchronous steps where pipelining cannot be used, introducing minor initial latency bubbles.

Future Directions: Video Models

- **Architectural Compatibility**: PipeFusion is natively compatible with modern video MM-DiTs models because they flatten spatial-temporal data into a unified sequence dimension
- **Superior Scalability for Video**: Video generation is an ideal domain for PipeFusion because video models are typically larger and involve longer sequences, which normally amplifies communication overhead in standard parallelism.